Introduction to Java

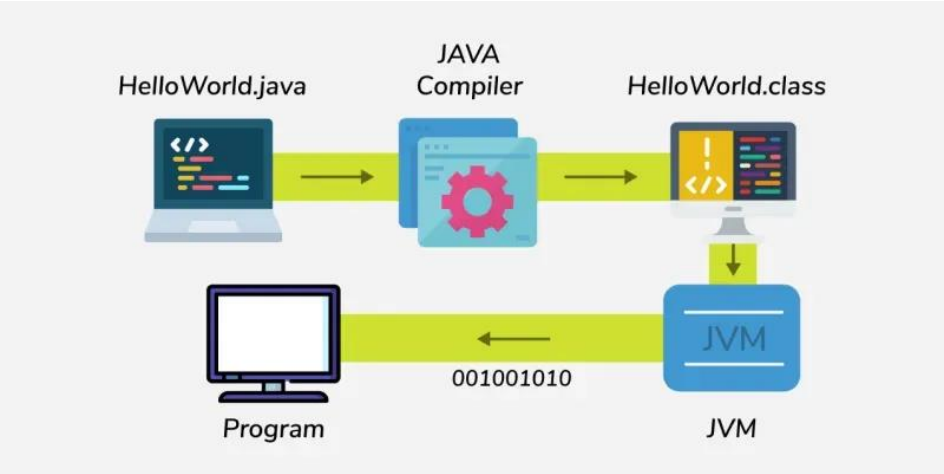
Java is a high-level, object-oriented programming language developed by **Sun Microsystems in 1995**. It is platform-independent, which means we can write code once and run it anywhere using the Java Virtual Machine (JVM). Java is mostly used for building **desktop applications, web applications, Android apps**, and **enterprise systems.**

Key Features of Java

- **Platform Independent:** Java is famous for its **Write Once, Run Anywhere (WORA) feature**. This means we can write our Java code once, and it will run on any device or operating system without changing anything.
- **Object-Oriented**: Java follows the object-oriented programming. This makes code clean and reusable.
- **Security**: Java does not support pointers, it includes built-in protections to keep our programs secure from common problems like memory leakage.
- **Multithreading**: Java programs can do many things at the same time using multiple threads. This is useful for handling complex tasks like processing transactions.
- **Just-In-Time (JIT) Compiler**: Java uses a JIT compiler. It improves performance by converting the bytecode into machine readable code at the time of execution.

Hello World Program in Java

```
public class HelloWorld
{
   public static void main(String[] args)
{
      System.out.println("Hello World!");
   }
}
```



Comments in Java

The comments are the notes written inside the code to explain what we are doing. The comment lines are not executed while we run the program.

Single-line comment:

// This is a comment

Multi-line comment:

```
/*
This is a multi-line comment.
This is useful for explaining larger sections of code.
*/
```

Naming Conventions in Java

- Java uses standard naming rules that make the code easier and improves the readability.
- In Java, the class names start with a capital letter for example, **HelloWorld**. Method and variable names start with a lowercase letter and use camelCase like **printMessage**.
- And the constants are written in all uppercase letters with underscores like **MAX_SIZE.**

Famous Applications Built Using Java

- **Android Apps**: Most of the Android mobile apps are built using Java.
- **Netflix**: This uses Java for content delivery and backend services.
- **Amazon**: Java language is used for its backend systems.
- **LinkedIn:** This uses Java for handling high traffic and scalability.
- **Minecraft**: This is one of the world's most popular games that is built in Java.
- **Spotify**: This uses Java in parts of its server-side infrastructure.
- **Uber**: Java is used for backend services like trip management.
- **NASA WorldWind**: This is a virtual globe software built using Java.

Differences Between JDK, JRE and JVM

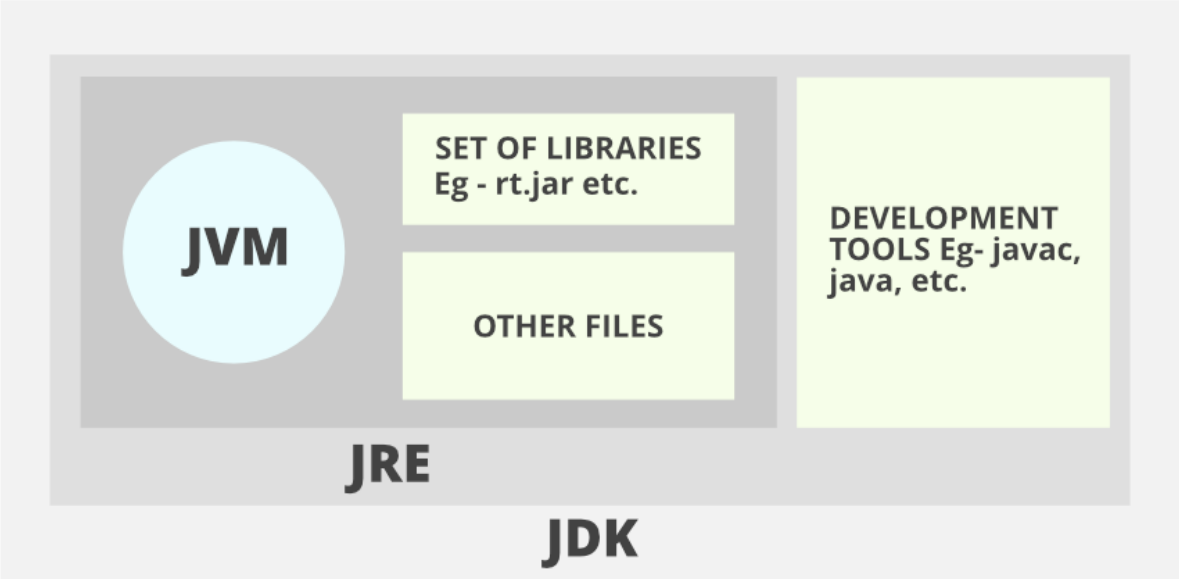
The main difference between JDK, JRE, and JVM is:

- JDK: **Java Development Kit** is a software development environment used for developing Java applications and applets.
- JRE: JRE stands for **Java Runtime Environment,** and it provides an environment to run only the Java program onto the system.
- JVM: JVM stands for **Java Virtual Machine** and is responsible for executing the Java program.

JDK vs JRE vs JVM

Aspect	JDK	JRE	JVM
Purpose	Used to develop Java applications	Used to run Java applications	Executes Java bytecode
Platform Dependency	Platform-dependent (OS specific)	Platform-dependent (OS specific)	JVM is OS-specific, but bytecode is platform-independent
Includes	JRE + Development tools (javac, debugger, etc.)	JVM + Libraries (e.g., rt.jar)	ClassLoader, JIT Compiler, Garbage Collector
Use Case	Writing and compiling Java code	Running a Java application on a system	Convert bytecode into native machine code

Note: *The JVM is platform-independent in the sense that the bytecode can run on any machine with a JVM, but the actual JVM implementation is platform-dependent. Different operating systems (e.g., Windows, Linux, macOS) require different JVM implementations that interact with the specific OS and hardware*



JDK (Java Development Kit)

The JDK is a software development kit that provides the environment to develop and execute the java application. It includes two things:

- Development Tools (to provide an environment to develop your java programs)
- JRE (to execute your java program)

Note:

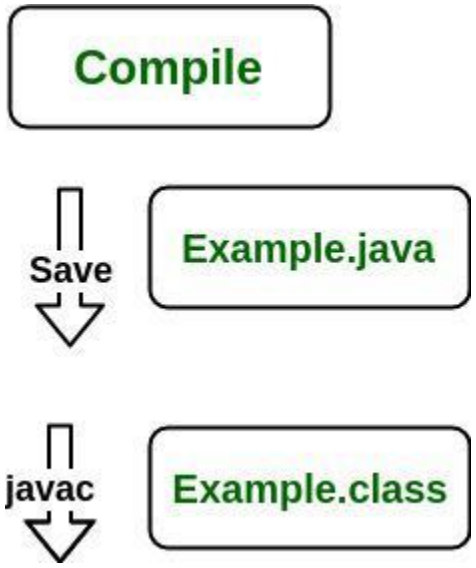
- *JDK is only for development (it is not needed for running Java programs)*

- *JDK is platform-dependent (different version for windows, Linux, macOS)*

Working of JDK

The JDK enables the development and execution of Java programs. Consider the following process:

- **Java Source File (e.g., Example.java)**: You write the Java program in a source file.
- **Compilation:** The source file is compiled by the Java Compiler (part of JDK) into bytecode, which is stored in a .class file (e.g., Example.class).
- **Execution:** The bytecode is executed by the JVM (Java Virtual Machine), which interprets the bytecode and runs the Java program.



The following actions occur at runtime as listed below:

- Class Loader
- Byte Code Verifier
- Interpreter
    - Execute the Byte Code
    - Make appropriate calls to the underlying hardware

JRE ((Java Runtime Environment)

The JRE is an installation package that provides an environment to **only run(not develop)** the Java program (or application) onto your machine. JRE is only used by those who only want to run Java programs that are end-users of your system.

Note:

- *JRE is only for end-users (not for developers).*
- *JRE is platform-dependent (different versions for different OS)*

Working of JRE

When you run a Java program, the following steps occur:

- **Class Loader**: The JRE's class loader loads the .class file containing the bytecode into memory.
- **Bytecode Verifier**: JRE includes a bytecode verifier to ensure security before execution
- **Interpreter**: JVM uses an interpreter + JIT compiler to execute bytecode for optimal performance
- **Execution**: The program executes, making calls to the underlying hardware and system resources as needed.

JVM (Java Virtual Machine)

The **JVM** is a very important part of both JDK and JRE because it is contained or inbuilt in both. Whatever Java program you run using JRE or JDK goes into JVM and JVM is responsible for executing the java program line by line, hence it is also known as an ***interpreter*.**

Note:

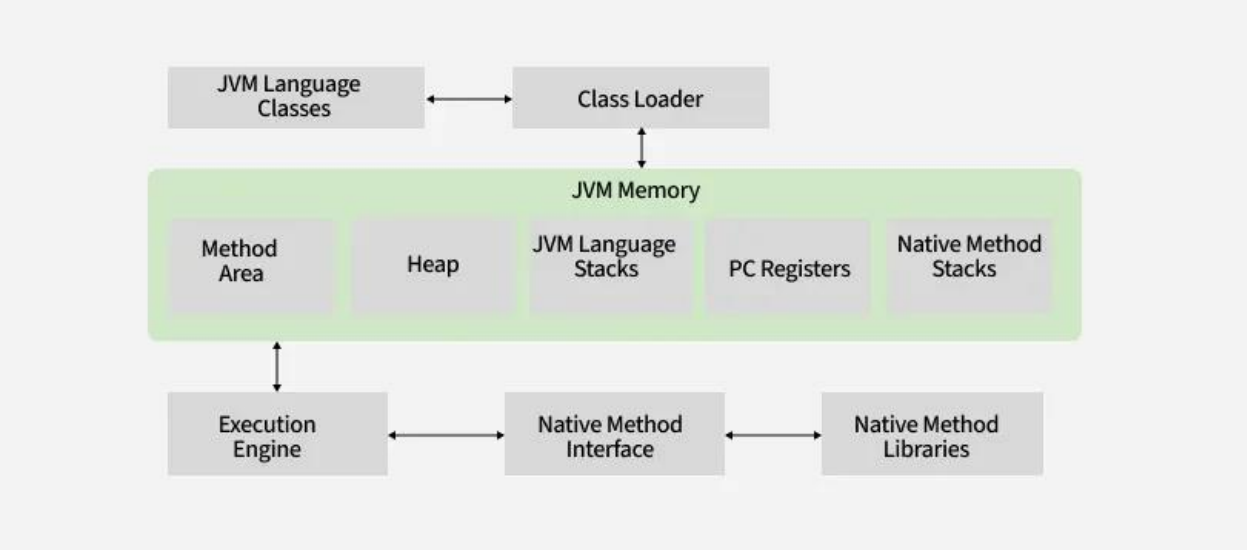
- *JVM is platform -dependent (different JVMs for window, linux, macOS).*
- *Bytecode (.class files) is platform-independent (same file runs in any JVM).*

- *While JVM includes an interpreter, modern implementations primarily use JIT compilation for faster execution*

Working of JVM

It is mainly responsible for three activities.

- Loading
- Linking
- Initialization



Java Identifiers

An **identifier in Java** is the name given to **Variables, Classes, Methods, Packages, Interfaces**, etc. These are the unique names used to identify programming elements. Every **Java Variable** must be identified with a unique name.

Example:

```
public class Test
{
public static void main(String[] args)
{
    int a = 20;
}
}
```

In the above Java code, we have 5 identifiers as follows:

- **Test**: Class Name
- **main**: Method Name
- **String**: Predefined Class Name
- **args**: Variable Name
- **a**: Variable Name

Rules For Naming Java Identifiers

There are certain rules for defining a valid Java identifier. These rules must be followed, otherwise, we get a compile-time error. These rules are also valid for other languages like C and C++.

- The only allowed characters for identifiers are all alphanumeric characters([**A-Z**],[**a-z**],[**0-9**]), '**$**'(dollar sign) and '**_**' (underscore). For example, "geek@" is not a valid Java identifier as it contains a '@', a special character.

- Identifiers should **not** start with digits(**[0-9]**). For example, "123geeks" is not a valid Java identifier.
- Java identifiers are **case-sensitive**.
- There is no limit on the length of the identifier, but it is advisable to use an optimum length of 4 - 15 letters only.
- **Reserved Words** can't be used as an identifier. For example, "int while = 20;" is an invalid statement as a while is a reserved word.

Note: Java has 53 reserved words (including 50 keywords and 3 literals), that are not allowed to be used as identifiers.

Examples of Valid Identifiers

MyVariable
MYVARIABLE
myvariable
x
i
x1
i1
_myvariable
$myvariable
sum_of_array
geeks123

Examples of Invalid Identifiers

My Variable // contains a space
123geeks // Begins with a digit
a+c // plus sign is not an alphanumeric character
variable-2 // hyphen is not an alphanumeric character
sum_&_difference // ampersand is not an alphanumeric character

Reserved Words in Java

Any programming language reserves some words to represent functionalities defined by that language. These words are called reserved words. They can be briefly categorized into two parts:

- K**eywords** (50): Keywords define functionalities.
- **literals** (3): Literals define value.

Identifiers are stored by **symbol tables** and used during the **lexical, syntax,** and **semantic** analysis phase of compilation.

List of Java Reserved Words

abstract	continue	for	protected	transient
Assert	Default	Goto	public	Try
Boolean	Do	If	Static	throws
break	double	implements	strictfp	Package
byte	else	import	super	Private
case	enum	Interface	short	switch

abstract	continue	for	protected	transient
Catch	Extends	instanceof	return	void
Char	Final	Int	synchronized	volatile
class	finally	long	throw	Date
const	float	Native	This	while

Note: The keywords **const** and **goto** are reserved, even though they are not currently used in Java. In place of const, the final keyword is used. Some keywords like strictfp are included in later versions of Java.

Java Data Types

Java is statically typed and also a strongly typed language because each type of data, such as integer, character, hexadecimal, packed decimal etc. is predefined as part of the programming language, and all constants or variables defined for a given program must be declared with the specific data types.

Data types in Java are of different sizes and values that can be stored in a variable that is made as per convenience and circumstances to handle different scenarios or data requirements.

Why Data Types Matter in Java?

Data types matter in Java because of the following reasons, which are listed below:

- **Memory Efficiency:** Choosing the right type (byte vs int) saves memory.
- **Performance:** Proper types reduce runtime errors.
- **Code Clarity:** Explicit typing makes code more readable.

Java Data Type Categories

Java has two categories in which data types are segregated

1. Primitive Data Type: These are the basic building blocks that store simple values directly in memory. Examples of primitive data types are

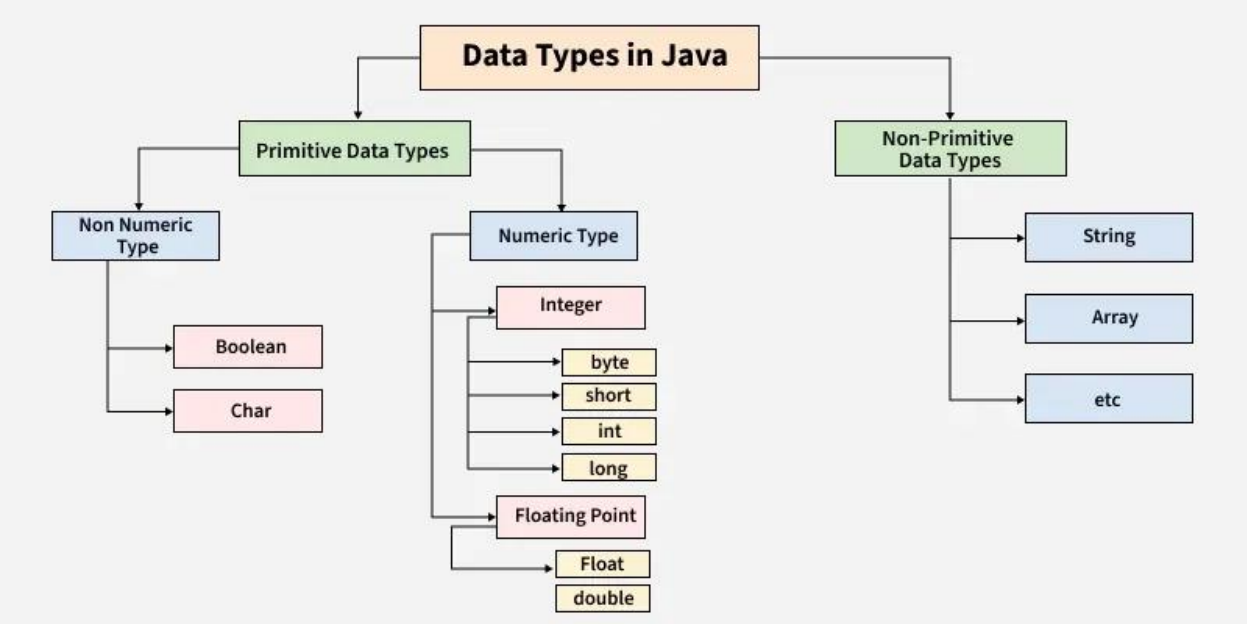
- **boolean**
- **char**
- **byte**
- **short**
- **int**
- **long**
- **float**
- **double**

Note: *The Boolean with uppercase B is a wrapper class for the primitive boolean type.*

2. Non-Primitive Data Types (Object Types): These are reference types that store memory addresses of objects. Examples of Non-primitive data types are

- **String**
- **Array**
- **Class**
- **Interface**
- **Object**

The below diagram demonstrates different types of primitive and non-primitive data types in Java.



Primitive Data Types in Java

Primitive data store only single values and have no additional capabilities. There are **8 primitive data types**. They are depicted below in tabular format below as follows:

Type	Description	Default	Size	Example Literals	Range of values
boolean	true or false	false	JVM-dependent (typically 1 byte)	true, false	true, false
byte	8-bit signed integer	0	1 byte	(none)	-128 to 127
char	Unicode character (16 bit)	\u0000	2 bytes	'a', '\u0041', '\101', '\\', '\', '\n', 'β'	0 to 65,535 (unsigned)
short	16-bit signed integer	0	2 bytes	(none)	-32,768 to 32,767
int	32-bit signed integer	0	4 bytes	-2,0,1	-2,147,483,648 to 2,147,483,647

Type	Description	Default	Size	Example Literals	Range of values
long	64-bit signed integer	0L	8 bytes	-2L,0L,1L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	32-bit IEEE 754 floating-point	0.0f	4 bytes	3.14f, -1.23e-10f	~6-7 significant decimal digits
double	64-bit IEEE 754 floating-point	0.0d	8 bytes	3.1415d, 1.23e100d	~15-16 significant decimal digits